



**Licenciatura em Engenharia e Gestão de Sistemas de Informação
Universidade do Minho**

Base de Dados II

2025/2026

**Momento 3
Grupo 5 - PL5**

Ana Catarina Pimenta Gomes - a104158

Filipe Meira de Sousa- a111056

Luana Daniela Pereira Ferreira - a106479

Nelson José Sousa Teixeira - a105131

Rui Pedro Teixeira de Oliveira - a110071



Índice

Introdução.....	2
Dados e Modelo do Domínio da Aplicação.....	3
Descrição Geral dos Dados.....	3
Modelo Geral dos Dados.....	8
Definição de Questões Analíticas Genéricas.....	9
Momento 1.....	11
Modelo dos Dados para Implementação.....	11
Questões Analíticas para Implementação.....	12
Implementação da Base de Dados.....	17
Momento 2.....	20
Modelo dos Dados para Implementação.....	20
Questões Analíticas para Implementação.....	21
Implementação da Base de Dados.....	25
Momento 3.....	28
Modelo dos Dados para Implementação.....	28
Questões Analíticas para Implementação.....	29
Implementação da Base de Dados.....	36
Síntese de Resultados MySQL.....	40
Síntese de Resultados Cassandra.....	41
Síntese de Resultados Neo4j.....	42
Avaliação do Grupo de Trabalho.....	43
Anexos.....	44
M1.....	44
M2.....	46
M3.....	48
Referências Bibliográficas.....	50

Introdução

O Momento 3 deste projeto visa a transição do modelo de dados anteriormente desenvolvido em *MySQL* e *Cassandra* para uma implementação utilizando o sistema de gestão de bases de dados *Neo4j*. Esta fase representa uma nova adaptação do modelo ao paradigma orientado a grafos, no qual os dados são organizados através de nós, relações e propriedades.

Nos momentos anteriores, o domínio do *Airbnb* foi inicialmente representado através de uma estrutura relacional, adequada à normalização dos dados e à utilização de junções, e posteriormente adaptado para *Cassandra*, onde o modelo passou a ser orientado às consultas e à desnormalização. No Momento 3, a abordagem passa a privilegiar a representação explícita das relações entre entidades, permitindo explorar de forma direta as ligações entre anfitriões, alojamentos, bairros, disponibilidade, avaliações e hóspedes.

A utilização do *Neo4j* permite modelar o domínio de forma próxima à realidade da plataforma *Airbnb*. Os anfitriões encontram-se ligados aos alojamentos que disponibilizam, os alojamentos pertencem a bairros, cada alojamento possui registos de disponibilidade e preço ao longo do tempo, e as avaliações são escritas por hóspedes e associadas aos respetivos alojamentos. Desta forma, consultas que anteriormente dependiam de junções ou de estruturas desnormalizadas passam a ser respondidas através da navegação pelo grafo.

Este momento envolve não apenas a criação dos nós e relações em *Neo4j*, mas também a adaptação das questões analíticas definidas nos momentos anteriores para a linguagem *Cypher*. As consultas foram reorganizadas de modo a explorar os percursos existentes no grafo, permitindo analisar padrões relacionados com preços, procura, disponibilidade, avaliações e comportamento dos hóspedes.

O principal objetivo deste momento é assegurar que o modelo de dados em *Neo4j* suporta as necessidades analíticas do *dataset* do *Airbnb*, mantendo coerência conceptual com os modelos anteriores e evidenciando as vantagens de uma base de dados orientada a grafos na análise de relações entre entidades. Nas secções seguintes, apresenta-se o modelo de dados para implementação, a adaptação das questões analíticas para *Cypher* e os principais passos realizados na implementação da base de dados.

Dados e Modelo do Domínio da Aplicação

Descrição Geral dos Dados

O dataset *Inside Airbnb* disponibiliza dados públicos recolhidos diretamente da plataforma *Airbnb*, com o objetivo de permitir análises independentes sobre oferta de alojamento, preços, disponibilidade, avaliações e impacto do alojamento local nas cidades. Os dados são atualizados periodicamente e organizados em vários ficheiros *CSV* (*Comma-Separated Values*), cada um representando uma dimensão distinta do ecossistema *Airbnb*.

A partir do *link* oficial, cada cidade contém os seguintes ficheiros principais:

- **listings.csv**: informação detalhada sobre cada anúncio;
- **calendar.csv**: disponibilidade diária e preço por data;
- **reviews.csv**: avaliações escritas pelos hóspedes;
- **neighbourhoods.csv**: lista de bairros da cidade;
- **listings_summary.csv** e **reviews_summary.csv**: versões resumidas (não usadas no modelo).

Embora o *dataset* não forneça diretamente entidades como Reserva ou Pagamento, estas podem ser inferidas a partir dos dados de disponibilidade e preços, permitindo construir um modelo conceptual coerente com o funcionamento real do *Airbnb*.

A seguir descreve-se cada conjunto de dados e como este se mapeia para as entidades definidas pelo grupo:

- **listings.csv**: Entidade Casa + parte da entidade Pessoa (anfitrião)

Este ficheiro contém um registo por anúncio e inclui atributos sobre:

- localização (bairro, latitude, longitude);
- características da casa (tipo, capacidade, comodidades);
- preço base;
- informação do anfitrião (*host_id*, *host_name*, *superhost*, ...).

Assim, este ficheiro alimenta duas entidades:

- Casa: atributos físicos e económicos do alojamento
- Pessoa (anfitrião): atributos do proprietário/anfitrião

- **calendar.csv**: Entidade Reserva (inferida)

O ficheiro *calendar* contém um registo por dia e por casa, indicando:

- preço para essa data;
- se está disponível ou não.

Como o *Airbnb* não disponibiliza reservas reais, o modelo conceptual interpreta:

- available = f: dia reservado;
- sequências de dias consecutivos reservados: uma reserva.

Este ficheiro permite construir:

- Reserva: período de estadia
- Pagamento: valor total calculado a partir dos preços diários

- **reviews.csv**: Entidade Avaliação

Contém avaliações escritas pelos hóspedes, incluindo:

- data;
- texto;
- identificador do hóspede;
- alojamento associado.

Este ficheiro alimenta diretamente a entidade Avaliação.

- **neighbourhoods.csv**: Atributo auxiliar da entidade Casa

Este ficheiro disponibiliza a nomenclatura oficial dos bairros da cidade, assegurando que a localização dos alojamentos é registada de forma consistente e padronizada na entidade Casa.



A seguir apresentam-se as tabelas de meta-dados caracterizadas para cada conjunto de dados, com atributos, significado, valores possíveis e exemplos reais:

- **listings.csv**

Atributo	Significado	Valores Possíveis	Exemplo
id	Identificador único atribuído ao anúncio na plataforma <i>Airbnb</i>	Inteiro positivo	17823451
host_id	Identificador único do anfitrião	Inteiro positivo	9234412
host_name	Nome público do anfitrião	Texto	“Jordi”
host_since	Data em que o anfitrião se registou na plataforma	AAAA-MM-DD	2016-04-18
host_is_superhost	Indica se o anfitrião possui estatuto de <i>Superhost</i>	t (<i>true</i>) / f (<i>false</i>)	t
neighbourhood	Bairro onde o alojamento está localizado	Texto	“Gràcia”
latitude / longitude	Latitude e longitude da localização do alojamento	Decimal	41.3892 / 2.1687
room_type	Tipo de espaço disponibilizado ao hóspede	<i>Entire home/apt, Private room, Shared room, Hotel room</i>	“ <i>Entire home/apt</i> ”
property_type	Tipo de propriedade física	<i>Apartment, Loft, Condominium, Studio, House, Guesthouse</i>	“ <i>Condominium</i> ”
accommodates	Nº máximo de hóspedes permitidos	Inteiro ≥ 1	3
price	Preço base por noite	Valor monetário	135
minimum_nights	Nº mínimo de noites exigido	Inteiro	2
number_of_reviews	Nº total de avaliações recebidas	Inteiro ≥ 0	87
review_scores_rating	Classificação média atribuída pelos hóspedes	0–100 ou nulo	96
availability_365	Nº de dias disponíveis no último ano	0–365	142



- calendar.csv

Atributo	Significado	Valores Possíveis	Exemplo
listing_id	Identificador da casa a que o registo diário pertence	Inteiro positivo	23145
date	Data específica do	AAAA-MM-DD	2024-06-15
available	Indica se o alojamento está disponível nessa data	t (<i>true</i>) / f (<i>false</i>)	f
price	Preço por noite aplicado nessa data	Valor monetário	145

- reviews.csv

Atributo	Significado	Valores Possíveis	Exemplo
listing_id	Identificador da casa avaliada	Inteiro positivo	17823451
id	Identificador único da avaliação	Inteiro positivo	552781992
date	Data em que a avaliação foi publicada	AAAA-MM-DD	2023-11-02
reviewer_id	Identificador do hóspede que avaliou	Inteiro positivo	88761234
reviewer_name	Nome único do hóspede	Texto	"Lúcia"
comments	Texto da avaliação escrita pelo hóspede	Texto livre	"Muito perto do metro. Excelente."

- neighbourhoods.csv

Atributo	Significado	Valores Possíveis	Exemplo
neighbourhood	Nome oficial do bairro onde o alojamento se situa	Texto	"El Raval"
neighbourhood_group	Distrito a que o bairro pertence	Texto	"Ciutat Vella"

- Entidade Pagamento (derivada)

Atributo	Significado	Valores Possíveis	Exemplo
pagamento_id	Identificador único do pagamento	Inteiro positivo	501
reserva_id	Identificador da reserva associada	Inteiro positivo	3001
valor_total	Valor total pago pela reserva (soma dos preços diários)	Valor monetário	540
método_pagamento	Método utilizado pelo hóspede para o pagamento	Cartão, <i>PayPal</i> , Transferência	“Cartão”
data_pagamento	Data em que o pagamento foi efetuado	AAAA-MM-DD	2024-07-20

Modelo Geral dos Dados

O Modelo Geral dos Dados representa a estrutura conceptual que caracteriza o domínio de aplicação do projeto, servindo como base para todas as fases de implementação. Este modelo é expresso através de um Diagrama Entidade-Relacionamento (*Chen, 1976*) e descreve as entidades principais, os seus atributos e os relacionamentos que emergem do funcionamento do *Airbnb*.

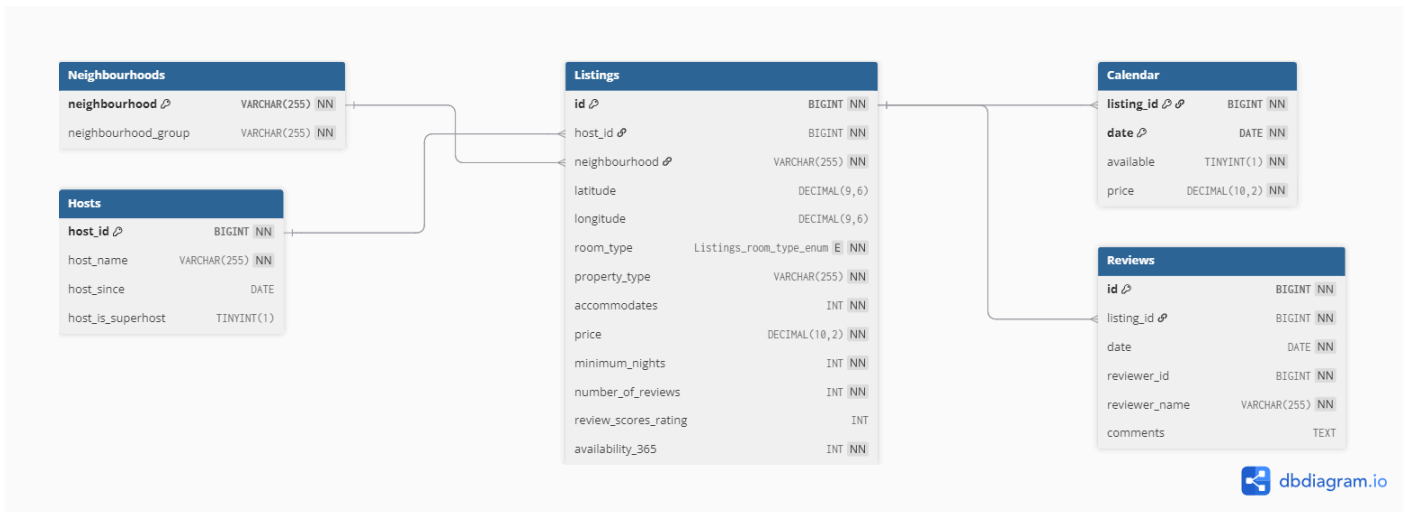


Figura 1 - Modelo de Dados do Domínio de Aplicação

Definição de Questões Analíticas Genéricas

A *Tabela 1* apresenta as questões analíticas definidas para orientar a análise dos dados ao longo dos três momentos do projeto, evidenciando padrões e tendências relevantes no domínio do *Airbnb*:

Questão	Dados e horizonte temporal a considerar
Como se distribuem os preços das casas por bairro e tipo de alojamento?	Tabelas: Listings, Neighbourhoods Atributos: Listings.price, Listings.room_type, Listings.property_type, Listings.neighbourhood_id, Neighbourhoods.neighbourhood_name Horizonte temporal: análise geral (preço atual) ou média anual se usar Calendar.price.
Qual a relação entre a classificação média das avaliações e o preço por noite?	Tabelas: Listings, Reviews Atributos: Listings.price, Listings.review_scores_rating, Listings.number_of_reviews, Reviews.listing_id Horizonte temporal: análise global ou evolução anual baseada em Reviews.date.
Que bairros apresentam maior concentração de casas e maior procura (dias reservados)?	Tabelas: Listings, Neighbourhoods, Calendar Atributos: Listings.id, Listings.neighbourhood_id, Calendar.available, Calendar.listing_id, Neighbourhoods.neighbourhood_name Horizonte temporal: número de dias reservados ao longo do ano usando Calendar.date.
Qual é o perfil dos anfitriões (Pessoa) em termos de número de casas, <i>superhost</i> e volume de avaliações?	Tabelas: Hosts, Listings, Reviews Atributos: Hosts.host_id, Hosts.host_is_superhost, Hosts.host_since, Listings.host_id, Listings.id, Reviews.listing_id Horizonte temporal: total acumulado ou evolução anual das reviews (Reviews.date).



Como evolui a disponibilidade e o preço ao longo do ano para diferentes tipos de casas?	Tabelas: Listings, Calendar Atributos: Calendar.date, Calendar.available, Calendar.price, Listings.room_type, Listings.property_type Horizonte temporal: análise mensal ou sazonal ao longo do ano.
Que fatores influenciam mais o valor total estimado de uma reserva (duração, preço por noite, época do ano)?	Tabelas: Listings, Calendar Atributos: Listings.price, Listings.minimum_nights, Calendar.price, Calendar.date, Listings.room_type Horizonte temporal: comparação por mês ou estação do ano.
Existe relação entre o comportamento dos hóspedes (avaliações) e o volume de reservas?	Tabelas: Reviews, Listings, Calendar Atributos: Reviews.reviewer_id, Reviews.date, Listings.number_of_reviews, Calendar.available Horizonte temporal: análise ao longo do tempo (por mês ou ano).
Qual é o impacto das características da casa (capacidade, tipo, localização) no preço e na procura?	Tabelas: Listings, Neighbourhoods, Calendar Atributos: Listings.accommodates, Listings.room_type, Listings.property_type, Listings.price, Listings.neighbourhood_id, Calendar.available Horizonte temporal: análise global ou anual.
Como se distribuem os pagamentos estimados por reserva ao longo do ano?	Tabelas: Calendar, Listings Atributos: Calendar.price, Calendar.date, Listings.minimum_nights Horizonte temporal: distribuição mensal ou sazonal ao longo do ano.
Existem padrões de comportamento entre hóspedes frequentes (Pessoa) e o tipo de casas que reservam?	Tabelas: Reviews, Listings Atributos: Reviews.reviewer_id, Reviews.listing_id, Listings.room_type, Listings.property_type, Reviews.date Horizonte temporal: histórico completo de reviews ou análise anual.

Tabela 1 - Questões Analíticas

Momento 1

Modelo dos Dados para Implementação

O modelo dos dados para implementação constitui a base estrutural que suporta o desenvolvimento dos três momentos do projeto (M1, M2 e M3). A partir do modelo conceptual do domínio, foi definido um modelo intermédio que organiza as entidades essenciais e os seus relacionamentos, servindo como referência comum para as diferentes tecnologias utilizadas. Este modelo é posteriormente transformado para formatos específicos (relacional, orientado a colunas e grafo), garantindo coerência conceptual e permitindo responder de forma eficiente às questões analíticas definidas.

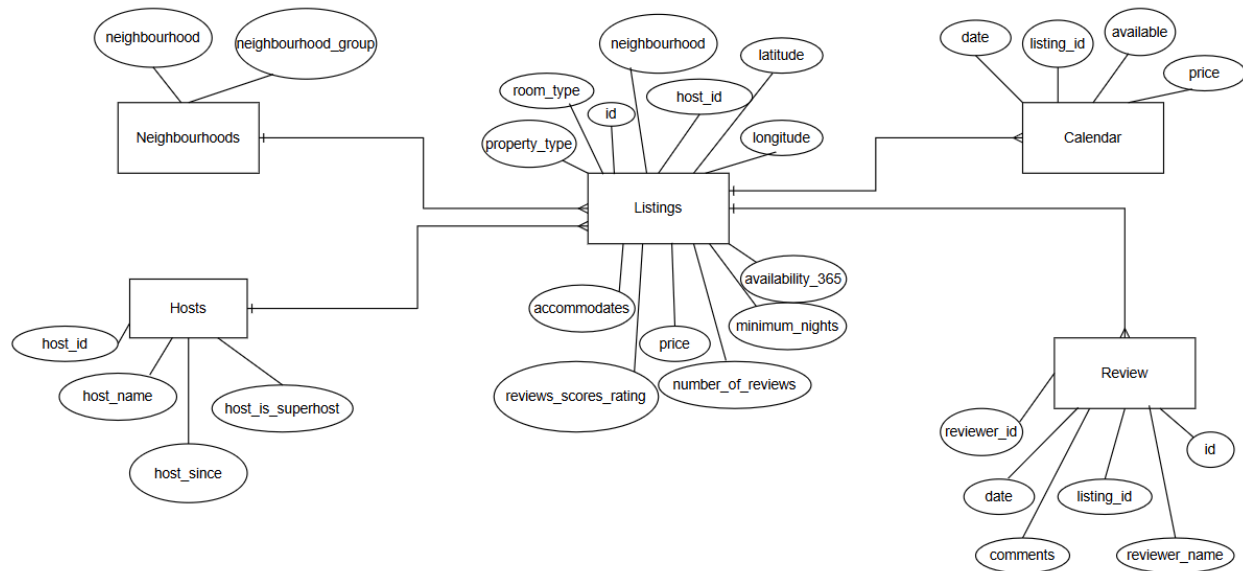


Figura 2 - Modelo de Dados para M1

Questões Analíticas para Implementação

As questões analíticas definidas na *Tabela 1* serão implementadas nos três momentos do projeto (M1, M2 e M3), recorrendo a diferentes modelos de dados e paradigmas tecnológicos. Embora as questões sejam conceptualmente as mesmas, a forma como são operacionalizadas varia consoante a tecnologia utilizada. Assim, nesta secção apresenta-se a adaptação das questões à linguagem e ao tipo de operações suportadas por cada momento:

• Momento 1 (M1) – MySQL (Modelo Relacional)

No M1, as questões são implementadas através de consultas SQL, recorrendo a operações como junções, agregações, filtros e ordenações. O modelo relacional permite responder às questões de forma estruturada e normalizada, garantindo integridade e consistência dos dados.

As questões a implementar em MySQL incluem:

- Distribuição de preços por bairro e tipo de alojamento;
- Relação entre classificação média e preço por noite;
- Identificação dos bairros com maior oferta e maior procura;
- Caracterização dos anfitriões (número de casas, *superhost*, avaliações);
- Evolução da disponibilidade e do preço ao longo do ano;
- Cálculo do valor total estimado das reservas;
- Relação entre comportamento dos hóspedes e volume de reservas;
- Impacto das características da casa no preço e na procura;
- Distribuição temporal dos pagamentos estimados;
- Padrões de comportamento de hóspedes frequentes.

Questão 1: Como se distribuem os preços das casas por bairro e tipo de alojamento?

```
SELECT
  l.neighbourhood,
  l.room_type,
  AVG(l.price) AS preco_medio,
  MIN(l.price) AS preco_minimo,
  MAX(l.price) AS preco_maximo,
  COUNT(*) AS total_casas
FROM Listings l
GROUP BY l.neighbourhood, l.room_type
ORDER BY l.neighbourhood, preco_medio DESC;
```

	AZ neighbourhood	AZ room_type	123 preco_medio	123 preco_minimo	123 preco_maximo	123 total_casas
1	Can Baró	Entire home/apt	158,956522	0	685	23
2	Can Baró	Private room	47,761905	0	145	21
3	Can Peguera	Private room	24	0	48	2
4	Canyelles	Entire home/apt	174	174	174	1
5	Ciutat Meridiana	Private room	49	42	56	2

Questão 2: Qual a relação entre a classificação média das avaliações e o preço por noite?

```
SELECT
    number_of_reviews,
    AVG(price) AS preco_medio,
    COUNT(*) AS total_casas
FROM Listings
GROUP BY number_of_reviews
ORDER BY number_of_reviews;
```

	123 number_of_reviews	123 preco_medio	123 total_casas
1	0	134,890158	4.989
2	1	114,764064	1.742
3	2	134,611434	1.032
4	3	113,35743	747
5	4	120,199319	587

Questão 3: Que bairros apresentam maior concentração de casas e maior procura (dias reservados)?

```
SELECT
    l.neighbourhood,
    COUNT(DISTINCT l.id) AS total_casas,
    SUM(CASE WHEN c.available = 0 THEN 1 ELSE 0 END) AS dias_reservados
FROM Listings l
JOIN Calendar c
    ON l.id = c.listing_id
GROUP BY l.neighbourhood
ORDER BY total_casas DESC, dias_reservados DESC;
```

	Az neighbourhood	123 total_casas	123 dias_reservados
1	la Dreta de l'Eixample	376	137.240
2	la Sagrada Família	234	85.410
3	la Vila de Gràcia	191	69.715
4	Sant Pere, Santa Caterina i la Ribera	173	63.145
5	el Raval	169	61.685

Questão 4: Qual é o perfil dos anfitriões (Pessoa) em termos de número de casas, superhost e volume de avaliações?

```
SELECT
    h.host_id,
    h.host_name,
    h.host_is_superhost,
    COUNT(DISTINCT l.id) AS total_casas,
    COUNT(r.id) AS total_avaliacoes
FROM Hosts h
LEFT JOIN Listings l
    ON h.host_id = l.host_id
LEFT JOIN Reviews r
    ON l.id = r.listing_id
GROUP BY h.host_id, h.host_name, h.host_is_superhost
ORDER BY total_casas DESC, total_avaliacoes DESC;
```

	123 host_id	AZ host_name	123 host_is_superhost	123 total_casas	123 total_avaliacoes
1	0		0	0	0
2	18.674	within an hour	0	0	0
3	223.096	N/A	0	0	0
4	242.335	within an hour	0	0	0
5	253.030	within an hour	0	0	0

Questão 5: Como evolui a disponibilidade e o preço ao longo do ano para diferentes tipos de casas?

```

SELECT
    YEAR(c.date) AS ano,
    MONTH(c.date) AS mes,
    l.room_type,
    AVG(c.price) AS preco_medio_diario,
    SUM(CASE WHEN c.available = 1 THEN 1 ELSE 0 END) AS dias_disponiveis,
    SUM(CASE WHEN c.available = 0 THEN 1 ELSE 0 END) AS dias_indisponiveis,
    ROUND(100 * AVG(CASE WHEN c.available = 1 THEN 1 ELSE 0 END), 2) AS taxa_disponibilidade_pct
FROM Calendar c
JOIN Listings l
    ON l.id = c.listing_id
GROUP BY YEAR(c.date), MONTH(c.date), l.room_type
ORDER BY ano, mes, l.room_type;

```

	123 ano	123 mes	AZ room_type	123 preco_medio_diario	123 dias_disponiveis	123 dias_indisponiveis	123 taxa_disponibilidade_pct
1	2.025	9	Entire home/apt	0	0	27.188	0
2	2.025	9	Private room	0	0	18.853	0
3	2.025	9	Shared room	0	0	114	0
4	2.025	9	Hotel room	0	0	224	0
5	2.025	10	Entire home/apt	0	0	52.111	0

Questão 6: Que fatores influenciam mais o valor total estimado de uma reserva (duração, preço por noite, época do ano)?

```

SELECT
    MONTH(date) AS mes,
    AVG(price) AS preco_medio,
    COUNT(CASE WHEN available = 0 THEN 1 END) AS noites_reservadas,
    SUM(CASE WHEN available = 0 THEN price ELSE 0 END) AS valor_total_estimado
FROM Calendar
GROUP BY mes
ORDER BY mes;

```

	123 mes	123 preco_medio	123 noites_reservadas	123 valor_total_estimado
1	1	0	89.063	0
2	2	0	80.444	0
3	3	0	89.063	0
4	4	0	86.190	0
5	5	0	89.063	0

Questão 7: Existe relação entre o comportamento dos hóspedes (avaliações) e o volume de reservas?

```
SELECT
    l.id AS casa,
    COUNT(DISTINCT r.id) AS total_avaliacoes,
    SUM(CASE WHEN c.available = 0 THEN 1 ELSE 0 END) AS dias_reservados
FROM Listings l
LEFT JOIN Reviews r
    ON l.id = r.listing_id
LEFT JOIN Calendar c
    ON l.id = c.listing_id
GROUP BY l.id
ORDER BY dias_reservados DESC;
```

	123 casa	123 total_avaliacoes	123 dias_reservados
1	24.424.717	1.820	664.300
2	24.343.358	1.387	506.255
3	18.572.961	1.140	416.100
4	24.193.951	1.038	378.870
5	72.436	1.011	369.015

Questão 8: Qual é o impacto das características da casa (capacidade, tipo, localização) no preço e na procura?

```
SELECT
    l.id AS casa,
    COUNT(DISTINCT r.id) AS total_avaliacoes,
    SUM(CASE WHEN c.available = 0 THEN 1 ELSE 0 END) AS dias_reservados
FROM Listings l
LEFT JOIN Reviews r
    ON l.id = r.listing_id
LEFT JOIN Calendar c
    ON l.id = c.listing_id
GROUP BY l.id
ORDER BY dias_reservados DESC;
```

	123 casa	123 total_avaliacoes	123 dias_reservados
1	24.424.717	1.820	664.300
2	24.343.358	1.387	506.255
3	18.572.961	1.140	416.100
4	24.193.951	1.038	378.870
5	72.436	1.011	369.015

Questão 9: Como se distribuem os pagamentos estimados por reserva ao longo do ano?

```
SELECT
    YEAR(date) AS ano,
    MONTH(date) AS mes,
    COUNT(CASE WHEN available = 0 THEN 1 END) AS noites_reservadas,
    SUM(CASE WHEN available = 0 THEN price ELSE 0 END) AS pagamento_total_estimado
FROM Calendar
GROUP BY ano, mes
ORDER BY ano, mes;
```


	123 ano	123 mes	123 noites_reservadas	123 pagamento_total_estimado
1	2.025	9	46.379	0
2	2.025	10	89.063	0
3	2.025	11	86.190	0
4	2.025	12	89.063	0
5	2.026	1	89.063	0

Questão 10: Existem padrões de comportamento entre hóspedes frequentes (Pessoa) e o tipo de casas que reservam?

```
SELECT
    r.reviewer_id,
    COUNT(r.id) AS total_avaliacoes,
    l.room_type,
    COUNT(*) AS vezes_reservado
FROM Reviews r
JOIN Listings l
    ON r.listing_id = l.id
GROUP BY r.reviewer_id, l.room_type
HAVING COUNT(r.id) > 5
ORDER BY total_avaliacoes DESC;
```

	123 reviewer_id	123 total_avaliacoes	AZ room_type	123 vezes_reservado
1	655.892.833	28	Entire home/apt	28
2	2.570.702	26	Entire home/apt	26
3	41.612.345	24	Entire home/apt	24
4	41.925.397	23	Entire home/apt	23
5	205.736	21	Entire home/apt	21

Implementação da Base de Dados

A implementação da base de dados foi realizada no âmbito do Momento 1, recorrendo ao sistema de gestão *DBeaver*. O processo seguiu o modelo dos dados para implementação definido anteriormente, envolvendo a criação da base de dados, das tabelas correspondentes às entidades do domínio, o carregamento e transformação dos dados provenientes dos ficheiros CSV do *Inside Airbnb*, e a execução das consultas necessárias para responder às questões analíticas identificadas. Nesta secção apresentam-se os principais passos da implementação, ilustrados com pequenos excertos de código, a listagem completa encontra-se em Anexos.

Criação e Preparação da Base de Dados

A implementação iniciou-se com a seleção da base de dados e com a eliminação de eventuais tabelas previamente existentes, garantindo que a execução do *script* ocorre num ambiente limpo e sem conflitos estruturais. Este procedimento permite assegurar a consistência do processo de criação das tabelas sempre que o *script* é executado novamente.

```
USE bd2;  
  
DROP TABLE IF EXISTS Calendar;  
DROP TABLE IF EXISTS Reviews;  
DROP TABLE IF EXISTS Listings;  
DROP TABLE IF EXISTS Hosts;  
DROP TABLE IF EXISTS Neighbourhoods;
```

Criação das Tabelas

Após a preparação da base de dados, procedeu-se à criação das tabelas que representam as entidades principais do domínio identificado: *Neighbourhoods*, *Hosts*, *Listings*, *Calendar* e *Reviews*.

A tabela *Neighbourhoods* armazena informação relativa aos bairros onde se localizam os alojamentos.

```
CREATE TABLE Neighbourhoods (  
  neighbourhood VARCHAR(255) NOT NULL,  
  neighbourhood_group VARCHAR(255) NOT NULL,  
  PRIMARY KEY (neighbourhood)  
);
```



A tabela *Hosts* representa os anfitriões responsáveis pelos anúncios na plataforma.

```
CREATE TABLE Hosts (  
  host_id BIGINT NOT NULL,  
  host_name VARCHAR(255) NOT NULL,  
  host_since DATE NULL,  
  host_is_superhost TINYINT(1) NULL,  
  PRIMARY KEY (host_id)  
);
```

A tabela *Listings* armazena a informação principal de cada anúncio, incluindo localização, características do alojamento e indicadores de avaliação. Esta tabela inclui ainda chaves estrangeiras que garantem a integridade referencial com as tabelas *Hosts* e *Neighbourhoods*.

```
CREATE TABLE Listings (  
  id BIGINT NOT NULL, -- listing id  
  host_id BIGINT NOT NULL,  
  neighbourhood VARCHAR(255) NOT NULL,  
  
  latitude DECIMAL(9,6) NULL,  
  longitude DECIMAL(9,6) NULL,  
  
  room_type ENUM('Entire home/apt', 'Private room', 'Shared room', 'Hotel room') NOT NULL,  
  property_type VARCHAR(255) NOT NULL,  
  
  accommodates INT NOT NULL,  
  price DECIMAL(10,2) NOT NULL,  
  minimum_nights INT NOT NULL,  
  
  number_of_reviews INT NOT NULL DEFAULT 0,  
  review_scores_rating INT NULL, -- 0..100 ou NULL  
  availability_365 INT NOT NULL, -- 0..365  
  
  PRIMARY KEY (id),  
  
  KEY idx_listings_host (host_id),  
  KEY idx_listings_neighbourhood (neighbourhood),  
  
  CONSTRAINT fk_listings_host  
  FOREIGN KEY (host_id) REFERENCES Hosts(host_id)  
  ON UPDATE CASCADE  
  ON DELETE RESTRICT,  
  
  CONSTRAINT fk_listings_neighbourhood  
  FOREIGN KEY (neighbourhood) REFERENCES Neighbourhoods(neighbourhood)  
  ON UPDATE CASCADE  
  ON DELETE RESTRICT  
);
```

Foram ainda criadas as tabelas *Calendar* e *Reviews*, responsáveis por armazenar, respetivamente, os dados de disponibilidade diária e as avaliações realizadas pelos hóspedes.

```
CREATE TABLE Calendar (  
  listing_id BIGINT NOT NULL,  
  date DATE NOT NULL,  
  available TINYINT(1) NOT NULL, -- 1=t, 0=f  
  price DECIMAL(10,2) NOT NULL,  
  
  PRIMARY KEY (listing_id, date),  
  KEY idx_calendar_date (date),  
  
  CONSTRAINT fk_calendar_listing  
  FOREIGN KEY (listing_id) REFERENCES Listings(id)  
  ON UPDATE CASCADE  
  ON DELETE CASCADE  
);
```

```
CREATE TABLE Reviews (  
  id BIGINT NOT NULL, -- review id  
  listing_id BIGINT NOT NULL,  
  date DATE NOT NULL,  
  reviewer_id BIGINT NOT NULL,  
  reviewer_name VARCHAR(255) NOT NULL,  
  comments TEXT NULL,  
  
  PRIMARY KEY (id),  
  KEY idx_reviews_listing_date (listing_id, date),  
  
  CONSTRAINT fk_reviews_listing  
  FOREIGN KEY (listing_id) REFERENCES Listings(id)  
  ON UPDATE CASCADE  
  ON DELETE CASCADE  
);
```

Carregamento e Transformação dos Dados

Após a criação da estrutura da base de dados, procedeu-se ao carregamento dos dados provenientes dos ficheiros CSV disponibilizados pelo dataset do *Inside Airbnb*. Para tal, foi utilizado o comando *LOAD DATA INFILE*, que permite importar grandes volumes de dados de forma eficiente.

Como exemplo, apresenta-se o carregamento da tabela *Calendar*, contendo informação diária sobre disponibilidade e preço dos alojamentos.

```
LOAD DATA INFILE '/var/lib/mysql/calendar.csv'
INTO TABLE Calendar
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(listing_id, date, available, price)
SET
available = IF(available='t',1,0),
price = REPLACE(price,'$', '');
```

Durante este processo foi também necessário realizar pequenas transformações nos dados. Por exemplo, os valores booleanos representados por t e f foram convertidos para 1 e 0, e os valores monetários tiveram o símbolo \$ removido para permitir a sua conversão para o tipo numérico definido na base de dados.

De forma semelhante, os dados dos anfitriões foram extraídos do ficheiro *listings.csv*, uma vez que o dataset não disponibiliza um ficheiro separado para esta entidade.

Verificação da Importação

Após a importação dos dados, foram executadas consultas simples para verificar se as tabelas foram corretamente preenchidas.

```
SELECT * FROM CALENDAR;
SELECT * FROM HOSTS;
SELECT * FROM LISTINGS;
SELECT * FROM NEIGHBOURHOODS;
SELECT * FROM REVIEWS;
```

Estas consultas permitiram confirmar a integridade e consistência dos dados carregados, garantindo que todas as entidades foram corretamente populadas antes da implementação das consultas analíticas.

Momento 2

Modelo dos Dados para Implementação

O modelo dos dados para implementação em *Cassandra* foi estruturado de forma a refletir as necessidades das consultas analíticas definidas para o projeto. Em vez de seguir uma organização baseada em entidades e relações, como no modelo relacional, as tabelas foram desenhadas para agrupar os atributos necessários a cada análise, garantindo acesso eficiente aos dados sem recorrer a operações de junção. A definição das *partition keys* e *clustering keys* foi orientada pelos padrões de acesso identificados, assegurando uma distribuição equilibrada da informação e tempos de resposta consistentes. O diagrama apresentado nesta secção ilustra a estrutura final das tabelas e a forma como estas suportam as consultas desenvolvidas no *Cassandra*.

Modelo Conceptual – Cassandra

LISTINGS_BY_NEIGHBOURHOOD		
PK	neighbourhood	text
	room_type	text
	listing_id	bigint
	price	decimal
	review_scores_rating	decimal
	number_of_reviews	int

HOSTS_PROFILE		
PK	host_id	bigint
	host_name	text
	host_is_superhost	boolean
	number_of_listings	int
	total_reviews	int
	host_since	timestamp

LISTINGS_FEATURES		
PK	year	int
PK	month	int
	accommodates	int
	room_type	text
	listing_id	bigint
	neighbourhood	text
	price	decimal
	available	boolean

LISTINGS_CALENDAR		
PK	year	int
PK	month	int
	neighbourhood	text
	room_type	text
	listing_id	bigint
	date	timestamp
	available	boolean
	minimum_nights	int

GUESTS_BEHAVIOR		
PK	guest_id	bigint
	guest_name	text
	total_reservations	int
	total_reviews	int
	average_review_score	decimal
	most_reserved_room_type	text

Figura 3 - Modelo de Dados para M2

Questões Analíticas para Implementação

Nesta fase, as questões analíticas definidas no início do projeto são implementadas em *Cassandra* através de um conjunto de tabelas desenhadas especificamente para suportar os padrões de acesso necessários. Como *Cassandra* não executa junções nem agregações distribuídas de forma eficiente, cada tabela foi construída para conter todos os atributos relevantes para um grupo de questões, garantindo que cada consulta pode ser respondida diretamente a partir de uma única estrutura. Assim, o foco passa a ser a organização dos dados em função das leituras que se pretendem otimizar, e não da representação das entidades do domínio.

A seguir apresenta-se a lógica de agrupamento das questões nas cinco tabelas criadas.

- Tabela 1: Questões 1 e 2

Esta tabela reúne apenas atributos estáticos das casas — bairro, tipo de alojamento, preço e métricas de avaliação. Como estes dados não variam ao longo do tempo, faz sentido armazená-los numa estrutura simples, particionada por *neighbourhood*, permitindo acesso imediato a todas as casas de um bairro sem necessidade de junções. A ordenação por *room_type* e *listing_id* dentro da partição garante consultas rápidas e organizadas sobre características e preços dos alojamentos.

Questão 1: Como se distribuem os preços das casas por bairro e tipo de alojamento?

```
SELECT neighbourhood, room_type, price
FROM listings by neighbourhood
WHERE neighbourhood = 'el Raval';
```

	123 review_scores_rating	123 number_of_reviews	123 price
12	5	4	17
13	5	8	17
14	4	6	16
15	4	5	17
16	5	2	15
17	5	14	0

Questão 2: Qual a relação entre a classificação média das avaliações e o preço por noite?

```
SELECT review_scores_rating, number_of_reviews, price
FROM listings by neighbourhood;
```

	123 accommodates	AZ room_type	AZ neighbourhood	123 price	AZ available
83	1	Entire home/apt	les Corts	0	false
84	1	Entire home/apt	les Corts	0	false
85	1	Entire home/apt	Hostafrancs	50	false
86	1	Entire home/apt	Hostafrancs	50	false
87	1	Entire home/apt	Hostafrancs	50	false
88	1	Entire home/apt	Hostafrancs	50	false

- Tabela 2: Questões 3, 5, 6 e 9

Esta tabela concentra todos os dados diários provenientes da junção entre *Calendar* e *Listings*. A partição por (*year*, *month*) agrupa os registos por período temporal, permitindo leituras eficientes sobre disponibilidade, preço e comportamento mensal. As *clustering keys*, *neighbourhood*, *room_type*, *listing_id* e *date*, organizam os dados dentro da partição de forma a suportar análises sequenciais e filtragens por localização, tipo de alojamento e dia, sem necessidade de operações adicionais.

Questão 3: Que bairros apresentam maior concentração de casas e maior procura (dias reservados)?

```
SELECT neighbourhood, listing_id, available
FROM listings_calendar
WHERE year = 2026 AND month = 2;
```

	AZ neighbourhood	123 listing_id	AZ available
222	Can Peguera	17.664.135	false
223	Can Peguera	17.664.135	false
224	Can Peguera	17.664.135	false
225	Diagonal Mar i el Front Mar	198.958	false
226	Diagonal Mar i el Front Mar	198.958	false
227	Diagonal Mar i el Front Mar	198.958	false

Questão 5: Como evolui a disponibilidade e o preço ao longo do ano para diferentes tipos de casas?

```
SELECT room_type, date, available, price
FROM listings_calendar
WHERE year = 2026 AND month = 2;
```

	AZ room_type	date	AZ available	123 price
26	Entire home/apt	2026-02-26	false	104
27	Entire home/apt	2026-02-27	false	104
28	Entire home/apt	2026-02-28	false	104
29	Entire home/apt	2026-02-01	false	125
30	Entire home/apt	2026-02-02	false	125
31	Entire home/apt	2026-02-03	false	125

Questão 6: Que fatores influenciam mais o valor total estimado de uma reserva (duração, preço por noite, época do ano)?

```
SELECT room_type, minimum_nights, available, price
FROM listings_calendar
WHERE year = 2026 AND month = 2;
```

	AZ room_type	123 minimum_nights	AZ available	123 price
26	Entire home/apt	4	false	104
27	Entire home/apt	4	false	104
28	Entire home/apt	4	false	104
29	Entire home/apt	3	false	125
30	Entire home/apt	3	false	125
31	Entire home/apt	3	false	125

Questão 9: Como se distribuem os pagamentos estimados por reserva ao longo do ano?

```
SELECT date, available, price
FROM listings_calendar
WHERE year = 2026 AND month = 2;
```

	date	A-Z available	123 price
26	2026-02-26	false	104
27	2026-02-27	false	104
28	2026-02-28	false	104
29	2026-02-01	false	125
30	2026-02-02	false	125
31	2026-02-03	false	125

- Tabela 3: Questão 4

Esta tabela foi desenhada para analisar o perfil dos anfitriões, reunindo num único registo os principais indicadores de cada *host*, como identificador, nome, estatuto de *superhost*, número de casas e total de avaliações.

A chave primária baseada em *host_id* permite acesso direto à informação de cada anfitrião, evitando junções entre *Hosts*, *Listings* e *Reviews*. Os dados foram previamente agregados em *MySQL* e carregados já consolidados em *Cassandra*, garantindo consultas eficientes.

Questão 4: Qual é o perfil dos anfitriões (Pessoa) em termos de número de casas, *superhost* e volume de avaliações?

```
SELECT host_id, host_name, superhost, total_casas, total_avaliacoes
FROM hosts_summary
ALLOW FILTERING;
```

	123 host_id	A-Z host_name	☒ superhost	123 total_casas	123 total_avaliacoes
1	928.941.163.244.537.984	A nice room in BCN	[]	0	0
2	21.722.173	Wide apartment in magical Barri Gotic area	[]	0	0
3	3.340.178	Cozy place for working in Barcelona / Les Corts	[]	0	0
4	1.081.012.326.072.502.840	Private triple ensuite room	[]	0	0
5	1.026.069.043.515.335.239	Spacious Double room	[]	0	0
6	641.020.247.716.930.473	VW Kombi T2 Westfalia disponible desde Barcelona	[]	0	0

- Tabela 4: Questão 8

Esta tabela combina características estruturais das casas (capacidade, tipo, bairro) com dados diários de preço e disponibilidade. A partição por (*year*, *month*) mantém a organização temporal, enquanto *accommodates*, *room_type* e *listing_id* permitem comparar procura e preço entre casas com diferentes capacidades e tipologias. Esta estrutura evita recorrer a dados totalmente diários quando apenas são necessárias combinações entre atributos fixos e indicadores de procura agregados.

Questão 8: Qual é o impacto das características da casa (capacidade, tipo, localização) no preço e na procura?

```
SELECT accommodates, room_type, neighbourhood, price, available
FROM listings_features_calendar
WHERE year = 2026 AND month = 2;
```

	AZ neighbourhood	AZ room_type	123 price
7	el Raval	Entire home/apt	218
8	el Raval	Entire home/apt	164
9	el Raval	Entire home/apt	65
10	el Raval	Entire home/apt	207
11	el Raval	Entire home/apt	99
12	el Raval	Entire home/apt	265

- Tabela 5: Questões 7 e 10

Esta tabela combina informação de avaliações, alojamentos e reservas estimadas, permitindo analisar a relação entre comportamento dos hóspedes e procura dos alojamentos.

Os dados foram previamente preparados em *MySQL* e armazenados de forma desnormalizada em *Cassandra*, incluindo atributos como tipo de alojamento, bairro, número de avaliações e volume estimado de reservas. Isto permite responder diretamente às *queries* sem necessidade de junções. A estrutura suporta tanto a análise por alojamento como a identificação de padrões de comportamento de hóspedes frequentes.

Questão 7: Existe relação entre o comportamento dos hóspedes (avaliações) e o volume de reservas?

```
SELECT listing_id, total_reviews_listing, total_reservas_listing
FROM reviews_reservas_comportamento
ALLOW FILTERING;
```

	123 listing_id	123 total_reviews_listing	123 total_reservas_listing
1	25.927.223	160	[NULL]
2	2.626.530	344	365
3	3.228.260	38	365
4	487.233	355	[NULL]
5	856.916.486.034.703.586	93	[NULL]
6	6.723.758	301	365

Questão 10: Existem padrões de comportamento entre hóspedes frequentes (Pessoa) e o tipo de casas que reservam?

```
SELECT reviewer_name, room_type, neighbourhood, total_reviews_reviewer, total_reviews_listing, total_reservas_listing
FROM reviews_reservas_comportamento
WHERE reviewer_id =189130;
```

	AZ reviewer_name	AZ room_type	AZ neighbourhood	123 total_reviews_reviewer	123 total_reviews_listing	123 total_reservas_listing
1	Melissa	Entire home/apt	el Barri Gòtic	1	25	365

Implementação da Base de Dados

A implementação da base de dados no Momento 2 consistiu na adaptação do modelo relacional desenvolvido no M1 para um modelo orientado a colunas em *Cassandra*. Esta fase exigiu uma reorganização completa da estrutura dos dados, uma vez que *Cassandra* não suporta junções nem agregações complexas, obrigando a que toda a informação necessária para cada consulta esteja previamente agregada e estruturada.

Ao contrário do M1, onde cada entidade do domínio originava uma tabela normalizada, no M2 o foco passou a ser as questões analíticas. Assim, em vez de criar uma tabela por entidade, foram criadas tabelas orientadas a grupos de questões, reduzindo o número total de estruturas e garantindo que cada tabela contém exatamente os atributos necessários para responder ao conjunto de *queries* associado.

Criação e Preparação do Keyspace

Tal como no M1 foi criado um esquema relacional em *MySQL*, no M2 iniciou-se por definir o *keyspace* responsável por agrupar todas as tabelas *Cassandra*. A estratégia de replicação utilizada foi adequada ao ambiente de desenvolvimento local, garantindo simplicidade e consistência durante os testes.

Após a criação do *keyspace*, procedeu-se à sua seleção e preparação para receber as tabelas necessárias ao projeto. Esta etapa corresponde, no contexto *Cassandra*, ao equivalente da criação da base de dados no M1.

```
USE bd2;  
DROP TABLE IF EXISTS listings_calendar;
```

Preparação dos Dados no MySQL

Os ficheiros CSV originais do *Inside Airbnb* foram carregados em *MySQL* tal como no M1, sem qualquer alteração ao seu conteúdo. A preparação dos dados consistiu exclusivamente na execução de consultas *SELECT* com *JOINS*, combinando informação das tabelas *Listings*, *Calendar*, *Reviews*, *Hosts* e *Neighbourhoods*.

Estas consultas foram construídas para reunir, numa única estrutura, todos os atributos necessários para cada conjunto de questões analíticas, permitindo gerar datasets consolidados que eliminam a necessidade de junções posteriores, uma vez que *Cassandra* não as suporta.



Os resultados destas consultas foram exportados diretamente para CSV e constituíram os ficheiros utilizados na carga das tabelas *Cassandra*.

```
SELECT
  YEAR(c.date) AS year,
  MONTH(c.date) AS month,
  l.neighbourhood,
  l.room_type,
  c.listing_id,
  c.date,
  CASE
    WHEN c.available = 1 THEN 'true'
    WHEN c.available = 0 THEN 'false'
    ELSE NULL
  END AS available,
  l.price,
  l.minimum_nights
FROM Calendar c
JOIN Listings l
  ON c.listing_id = l.id;
```

Criação das Tabelas em Cassandra

Ao contrário do modelo relacional do M1, onde cada entidade originava uma tabela normalizada, no M2 as tabelas foram desenhadas com base no agrupamento das questões analíticas. Cada grupo de questões partilhava o mesmo padrão de acesso aos dados, pelo que foi criada uma única tabela *wide-column* por grupo, reduzindo o número total de tabelas e evitando redundância.

As tabelas foram definidas com *partition keys* e *clustering keys* adequadas ao tipo de consulta que iriam suportar, garantindo eficiência na leitura e distribuição equilibrada dos dados no cluster. A desnormalização aplicada foi intencional e necessária, refletindo o paradigma orientado a queries de *Cassandra*.

```
CREATE TABLE IF NOT EXISTS listings_calendar(
  year INT,
  month INT,
  neighbourhood TEXT,
  room_type TEXT,
  listing_id BIGINT,
  date DATE,
  available BOOLEAN,
  price DECIMAL,
  minimum_nights INT,
  PRIMARY KEY ((year, month), neighbourhood, room_type, listing_id, date)
);
```

As **questões 1 e 2** foram agrupadas por utilizarem apenas atributos estáticos dos alojamentos, como bairro, tipo de alojamento, preço e métricas de avaliação. Como estas análises não dependem de variação temporal, faz sentido armazenar estes dados numa única tabela particionada por *neighbourhood*, permitindo consultar diretamente todos os alojamentos de um bairro e comparar preços e avaliações sem necessidade de junções ou dados adicionais.

As **questões 3, 5, 6 e 9** foram agrupadas por dependerem de dados diários de disponibilidade e preço, exigindo análise ao longo do tempo. Todas envolvem leitura sequencial por períodos (mês/ano) e cruzamento com localização e tipo de alojamento. Por isso, foi definida uma



partição por (*year*, *month*), permitindo filtrar diretamente intervalos temporais e suportar análises de evolução, procura e estimativas de reservas sem necessidade de percorrer toda a tabela.

A **questão 4** foi isolada numa tabela própria por se focar exclusivamente no perfil dos anfitriões, sendo centrada na entidade *host* e não em alojamentos ou datas. Como a análise é feita por anfitrião, a chave de partição *host_id* permite acesso direto a todos os indicadores relevantes de cada *host*, sem necessidade de considerar dimensões temporais ou geográficas.

A **questão 8** foi colocada numa tabela dedicada por combinar características estruturais dos alojamentos (como capacidade, tipo e localização) com indicadores de procura e preço. Embora utilize dados temporais, o foco principal é a comparação entre diferentes tipos de casas, justificando uma estrutura que privilegia atributos como *accommodates* e *room_type*, permitindo análises diretas sem recorrer a dados excessivamente detalhados.

As **questões 7 e 10** foram agrupadas por ambas analisarem o comportamento associado às avaliações e a sua relação com o volume de reservas. Estas análises exigem dados já agregados por alojamento, incluindo número de avaliações e indicadores de procura, pelo que foi criada uma tabela centrada em *listing_id*, permitindo observar diretamente padrões de utilização e comportamento sem necessidade de junções entre *Reviews*, *Listings* e *Calendar*.

Carregamento e Transformação dos Dados

Após a criação das tabelas, os CSVs derivados dos *SELECTs* do *MySQL* foram carregados diretamente para *Cassandra*. Cada ficheiro correspondia a um grupo de questões e continha apenas os atributos relevantes para a tabela associada. O processo de importação foi realizado de forma sequencial, garantindo que os dados eram inseridos corretamente e respeitavam a estrutura definida no *keyspace*.

```
SELECT COUNT(*) FROM listings_calendar;
```

Verificação da Importação

Tal como no M1, foi realizada uma fase de verificação após o carregamento dos dados. Foram executadas consultas simples sobre cada tabela *Cassandra*, com o objetivo de confirmar:

- a presença e consistência dos registos,
- a correta aplicação das chaves de partição e ordenação,
- a correspondência entre os dados carregados e os resultados obtidos no *MySQL*,
- e a integridade dos *datasets* gerados para cada grupo de questões.

Estas verificações incluíram leituras parciais das tabelas, contagens de registos e consultas filtradas por chaves específicas.

```
SELECT *  
FROM listings_calendar  
WHERE year = 2026 AND month = 2  
LIMIT 10;
```

Momento 3

Modelo dos Dados para Implementação

No Momento 3, a base de dados foi implementada em *Neo4j*, seguindo um modelo orientado a grafos. Neste modelo, as principais entidades do domínio *Airbnb* foram representadas como nós e as associações entre elas foram representadas como relações.

O modelo implementado é composto pelos seguintes nós: *Host*, *Listing*, *Neighbourhood*, *CalendarDay*, *Review* e *Guest*. O nó *Host* representa o anfitrião, *Listing* representa o alojamento, *Neighbourhood* representa o bairro, *CalendarDay* guarda a disponibilidade e preço diário, *Review* representa a avaliação e *Guest* representa o hóspede que escreveu a avaliação.

As relações criadas foram:

```
(Host)-[:HOSTS]->(Listing)
(Listing)-[:IN_NEIGHBOURHOOD]->(Neighbourhood)
(Listing)-[:HAS_CALENDAR]->(CalendarDay)
(Guest)-[:WROTE]->(Review)
(Review)-[:REVIEWS]->(Listing)
```

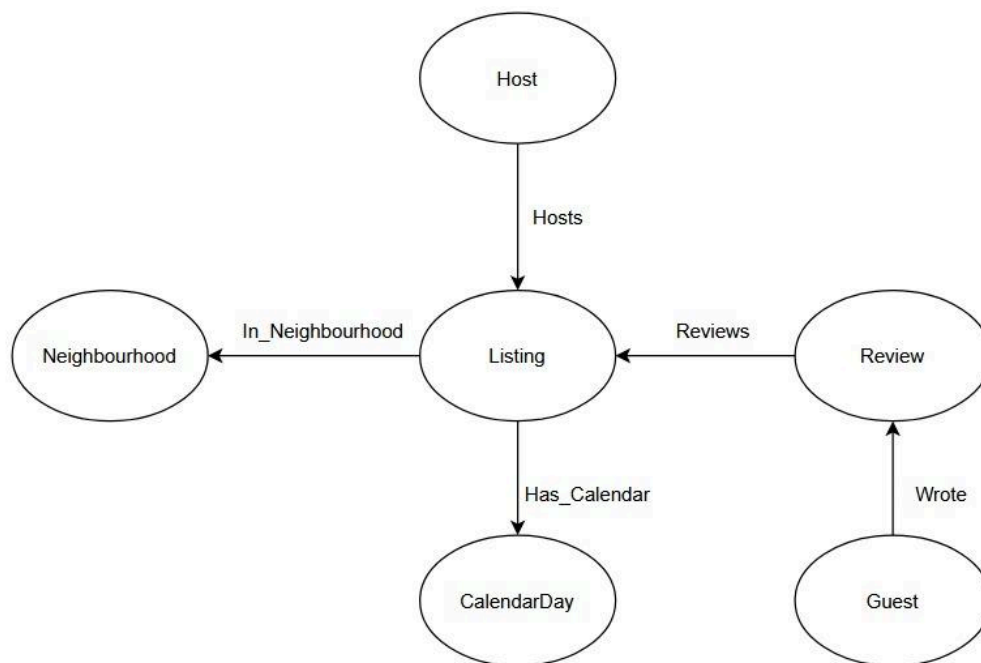


Figura 4 - Modelo de Dados para M3

Questões Analíticas para Implementação

No Momento 3, as questões analíticas foram implementadas em *Cypher*, a linguagem de consulta utilizada pelo *Neo4j*. As questões mantêm a mesma lógica definida nos momentos anteriores, mas foram adaptadas ao modelo de grafo, explorando diretamente as relações entre nós como *Host*, *Listing*, *Neighbourhood*, *CalendarDay*, *Review* e *Guest*.

Questão 1: Como se distribuem os preços das casas por bairro e tipo de alojamento?

A primeira questão analítica permite analisar a distribuição dos preços dos alojamentos por bairro e tipo de alojamento. Para isso, são considerados os nós *Listing*, agrupando os resultados por *neighbourhood* e *room_type*, e calculando o preço médio, mínimo e máximo.

```
MATCH (l:Listing)
RETURN l.neighbourhood AS neighbourhood,
       l.room_type AS room_type,
       round(avg(l.price), 2) AS preco_medio,
       min(l.price) AS preco_minimo,
       max(l.price) AS preco_maximo,
       count(l) AS total_casas
ORDER BY neighbourhood, preco_medio DESC;
```

neighbourhood	room_type	preco_medio	preco_minimo	preco_maximo	total_casas
"Can Baró"	"Entire home/apt"	158.96	0.0	685.0	23
"Can Baró"	"Private room"	47.76	0.0	145.0	21
"Can Peguera"	"Private room"	24.0	0.0	48.0	2
"Canyelles"	"Entire home/apt"	174.0	174.0	174.0	1
"Ciutat Meridiana"	"Private room"	49.0	42.0	56.0	2
"Diagonal Mar i el Front Marítim del Poblenou"	"Entire home/apt"	252.11	0.0	2000.0	119

Questão 2: Qual a relação entre a classificação média das avaliações e o preço por noite?

A segunda questão procura relacionar o preço por noite com os indicadores de avaliação disponíveis. Como nem todos os registos continham classificação média preenchida, foi usada a variável *number_of_reviews* como indicador do volume de avaliações, permitindo comparar o preço médio dos alojamentos com a média de *reviews* por bairro e tipo de alojamento.

```
MATCH (l:Listing)
WHERE l.price IS NOT NULL
RETURN l.neighbourhood AS bairro,
       l.room_type AS tipo_alojamento,
       round(avg(l.price), 2) AS preco_medio,
       round(avg(l.number_of_reviews), 2) AS media_reviews,
       count(l) AS total_casas
ORDER BY bairro, preco_medio DESC;
```

bairro	tipo_alojamento	preco_medio	media_reviews	total_casas
"Can Baró"	"Entire home/apt"	158.96	85.04	23
"Can Baró"	"Private room"	47.76	42.57	21
"Can Peguera"	"Private room"	24.0	21.5	2
"Canyelles"	"Entire home/apt"	174.0	56.0	1
"Ciutat Meridiana"	"Private room"	49.0	1.5	2
"Diagonal Mar i el Front Marítim del Poblenou"	"Entire home/apt"	252.11	78.82	119

Questão 3: Que bairros apresentam maior concentração de casas e maior procura (dias reservados)?

A terceira questão identifica os bairros com maior concentração de casas e maior procura. Esta análise combina os nós *Listing* e *CalendarDay*, utilizando a relação *HAS_CALENDAR* para contabilizar os dias indisponíveis, interpretados como dias reservados.

```
MATCH (l:Listing)
OPTIONAL MATCH (l)-[:HAS_CALENDAR]->(c:CalendarDay)
WITH l.neighbourhood AS neighbourhood,
     count(DISTINCT l) AS total_casas,
     sum(CASE WHEN c.available = false THEN 1 ELSE 0 END) AS dias_reservados
RETURN neighbourhood, total_casas, dias_reservados
ORDER BY total_casas DESC, dias_reservados DESC;
```


neighbourhood	total_casas	dias_reservados
<i>null</i>	38820	0
"la Dreta de l'Eixample"	2416	1131
"el Raval"	1519	173
"el Barri Gòtic"	1253	125
"Sant Pere, Santa Caterina i la Ribera"	1244	141
"la Vila de Gràcia"	1136	764

Questão 4: Qual é o perfil dos anfitriões (Pessoa) em termos de número de casas, *superhost* e volume de avaliações?

A quarta questão caracteriza o perfil dos anfitriões, analisando o número de alojamentos associados a cada *Host*, o estatuto de *superhost* e o volume de avaliações recebidas pelos seus alojamentos. Esta análise é feita através das relações *HOSTS* e *REVIEWS*.

```
MATCH (h:Host)-[:HOSTS]->(l:Listing)
OPTIONAL MATCH (l)-[:REVIEWS]-(r:Review)
RETURN h.host_id AS host_id,
       h.host_name AS host_name,
       h.host_is_superhost AS superhost,
       count(DISTINCT l) AS total_casas,
       count(DISTINCT r) AS total_avaliacoes
ORDER BY total_casas DESC, total_avaliacoes DESC;
```

host_id	host_name	superhost	total_casas	total_avaliacoes
346367515	"Ukio"	false	514	0
357946540	"Adria"	false	426	0
1447144	"Jacob"	false	413	41
32037490	"SweetInn"	false	257	0
221480824	"Alvaro"	false	244	0
21726991	"Silvia"	false	234	0

Questão 5: Como evolui a disponibilidade e o preço ao longo do ano para diferentes tipos de casas?

A quinta questão analisa a evolução da disponibilidade e do preço ao longo do ano, agrupando os dados por ano, mês e tipo de alojamento. Para isso, são usados os nós *CalendarDay*, que armazenam a data, preço diário e disponibilidade.

```
MATCH (l:Listing)-[:HAS_CALENDAR]->(c:CalendarDay)
RETURN c.year AS ano,
       c.month AS mes,
       l.room_type AS room_type,
       round(avg(c.price), 2) AS preco_medio_diario,
       sum(CASE WHEN c.available = true THEN 1 ELSE 0 END) AS dias_disponiveis,
       sum(CASE WHEN c.available = false THEN 1 ELSE 0 END) AS dias_indisponiveis,
       round(100.0 * avg(CASE WHEN c.available = true THEN 1.0 ELSE 0.0 END), 2) AS taxa_disponibilidade_pct
ORDER BY ano, mes, room_type;
```

ano	mes	room_type	preco_medio_diario	dias_disponiveis	dias_indisponiveis	taxa_disponibilidade_pct
2025	9	"Entire home/apt"	0.0	242	698	25.74
2025	9	"Private room"	0.0	47	133	26.11
2025	10	"Entire home/apt"	0.0	627	1171	34.87
2025	10	"Private room"	0.0	65	276	19.06
2025	11	"Entire home/apt"	0.0	1109	631	63.74
2025	11	"Private room"	0.0	120	210	36.36

Questão 6: Que fatores influenciam mais o valor total estimado de uma reserva (duração, preço por noite, época do ano)?

A sexta questão estima fatores associados ao valor total de uma reserva, considerando o preço médio por noite, o número mínimo de noites e os dias reservados. Esta análise permite observar diferenças por mês e tipo de alojamento.

```
MATCH (l:Listing)-[:HAS_CALENDAR]->(c:CalendarDay)
RETURN c.year AS ano,
       c.month AS mes,
       l.room_type AS room_type,
       round(avg(c.price), 2) AS preco_medio,
       round(avg(l.minimum_nights), 2) AS min_nights_medio,
       sum(CASE WHEN c.available = false THEN 1 ELSE 0 END) AS noites_reservadas,
       round(sum(CASE WHEN c.available = false THEN c.price ELSE 0 END), 2) AS valor_total_estimado
ORDER BY ano, mes, valor_total_estimado DESC;
```

ano	mes	room_type	preco_medio	min_nights_medio	noites_reservadas	valor_total_estimado
2025	9	"Entire home/apt"	0.0	10.1	698	0.0
2025	9	"Private room"	0.0	16.34	133	0.0
2025	10	"Entire home/apt"	0.0	10.14	1171	0.0
2025	10	"Private room"	0.0	16.0	276	0.0
2025	11	"Entire home/apt"	0.0	10.14	631	0.0
2025	11	"Private room"	0.0	16.0	210	0.0

Questão 7: Existe relação entre o comportamento dos hóspedes (avaliações) e o volume de reservas?

A sétima questão relaciona o comportamento dos hóspedes, medido pelo número de avaliações, com o volume de reservas estimado, medido pelos dias indisponíveis no calendário.

```
MATCH (l:Listing)
OPTIONAL MATCH (l)-[:REVIEWS]-(r:Review)
OPTIONAL MATCH (l)-[:HAS_CALENDAR]->(c:CalendarDay)
WITH l,
| count(DISTINCT r) AS total_avaliacoes,
| sum(CASE WHEN c.available = false THEN 1 ELSE 0 END) AS dias_reservados
RETURN l.listing_id AS casa,
| total_avaliacoes,
| dias_reservados
ORDER BY dias_reservados DESC;
```

casa	total_avaliacoes	dias_reservados
272282	436	153472
72436	1011	112221
273932	357	109599
197267	364	100464
330766	311	99520
273229	202	70498

Questão 8: Qual é o impacto das características da casa (capacidade, tipo, localização) no preço e na procura?

A oitava questão avalia o impacto das características da casa, como capacidade, tipo de alojamento, bairro e preço base, na procura estimada, usando novamente os dias reservados como indicador de procura.

```
MATCH (l:Listing)
OPTIONAL MATCH (l)-[:HAS_CALENDAR]->(c:CalendarDay)
WITH l,
| sum(CASE WHEN c.available = false THEN 1 ELSE 0 END) AS dias_reservados
RETURN l.listing_id AS casa,
| l.accommodates AS accommodates,
| l.room_type AS room_type,
| l.neighbourhood AS neighbourhood,
| l.price AS preco_base,
| dias_reservados
ORDER BY dias_reservados DESC;
```

casa	accommodates	room_type	neighbourhood	preco_base	dias_reservados
190348	0	"Private room"	"Pedraibes"	0.0	365
223096	0	"Entire home/apt"	"Sants"	0.0	365
253030	0	"Private room"	"Navas"	0.0	365
323854	0	"Private room"	"la Sagrada Família"	0.0	365
83283	0	"Private room"	"Sant Antoni"	82.0	359
144471	0	"Private room"	"Sant Antoni"	81.0	359

Questão 9: Como se distribuem os pagamentos estimados por reserva ao longo do ano?

A nona questão calcula a distribuição dos pagamentos estimados ao longo do ano. Como o dataset não possui pagamentos reais, o valor estimado é calculado através da soma dos preços dos dias em que o alojamento aparece como indisponível.

```
MATCH (:Listing)-[:HAS_CALENDAR]->(c:CalendarDay)
RETURN c.year AS ano,
| c.month AS mes,
| sum(CASE WHEN c.available = false THEN 1 ELSE 0 END) AS noites_reservadas,
| round(sum(CASE WHEN c.available = false THEN c.price ELSE 0 END), 2) AS pagamento_total_estimado
ORDER BY ano, mes;
```

ano	mes	noites_reservadas	pagamento_total_estimado
2025	9	831	0.0
2025	10	1447	0.0
2025	11	841	0.0
2025	12	675	0.0
2026	1	921	0.0
2026	2	880	0.0

Questão 10: Existem padrões de comportamento entre hóspedes frequentes (Pessoa) e o tipo de casas que reservam?

Por fim, a décima questão analisa padrões de comportamento entre hóspedes e tipos de alojamento avaliados. Para isso, são usadas as relações *WROTE* e *REVIEWS*, ligando hóspedes, avaliações e alojamentos.

```
MATCH (g:Guest)-[:WROTE]->(r:Review)-[:REVIEWS]->(l:Listing)
WITH g, l.room_type AS tipo_alojamento, count(r) AS total_avaliacoes
WHERE total_avaliacoes >= 1
RETURN gReviewer_id AS reviewer_id,
       gReviewer_name AS nome_hospede,
       tipo_alojamento,
       total_avaliacoes
ORDER BY total_avaliacoes DESC
LIMIT 50;
```

reviewer_id	nome_hospede	tipo_alojamento	total_avaliacoes
396372	"Kay"	"Entire home/apt"	5
599752242	"Katherine"	"Entire home/apt"	4
17564523	"Oliver"	"Entire home/apt"	4
2467186	"Christian"	"Private room"	4
26580568	"David"	"Entire home/apt"	4
27467001	"Daniel"	"Private room"	3

Implementação da Base de Dados

A implementação da base de dados no Momento 3 foi realizada em *Neo4j*, recorrendo a um ambiente local executado através de um container *Docker*. Esta fase teve como objetivo transformar os dados do *dataset Airbnb* num modelo orientado a grafos, criando os nós, relações, propriedades, *constraints* e índices necessários para responder às questões analíticas definidas.

Ao contrário dos momentos anteriores, em que os dados foram organizados em tabelas relacionais ou em tabelas orientadas às consultas, no *Neo4j* a implementação centrou-se na criação de uma rede de entidades relacionadas. Assim, os dados foram importados a partir dos ficheiros CSV e convertidos em nós como *Host*, *Listing*, *Neighbourhood*, *CalendarDay*, *Review* e *Guest*, ligados por relações como *HOSTS*, *IN_NEIGHBOURHOOD*, *HAS_CALENDAR*, *WROTE* e *REVIEWS*.

Criação e Preparação da Base de Dados em Grafo

A primeira etapa consistiu na preparação da base de dados em *Neo4j*. Para isso, foram criadas *constraints* e índices que permitem garantir a consistência dos dados e melhorar o desempenho das consultas.

As *constraints* foram aplicadas aos principais identificadores do modelo, assegurando que não existiam entidades duplicadas, como dois alojamentos com o mesmo *listing_id*, dois anfitriões com o mesmo *host_id* ou duas avaliações com o mesmo *review_id*. Foram também criados índices sobre propriedades frequentemente utilizadas nas consultas, como o bairro, o tipo de alojamento, o ano e mês do calendário e a data das avaliações.

Esta preparação foi importante para garantir que a importação dos dados ocorria de forma controlada, evitando duplicações e permitindo consultas mais eficientes na fase analítica.

```
CREATE CONSTRAINT host_id IF NOT EXISTS  
FOR (h:Host) REQUIRE h.host_id IS UNIQUE;
```

Importação dos Dados

Após a criação das *constraints* e índices, procedeu-se à importação dos dados. Como o *Neo4j* estava a ser executado num container *Docker*, os ficheiros CSV foram colocados na pasta *import* do *container*. Desta forma, foi possível utilizar o comando *LOAD CSV WITH HEADERS* para ler diretamente os ficheiros e criar os nós e relações necessários.



A importação foi realizada a partir dos ficheiros *neighbourhoods.csv*, *listings.csv*, *calendar.csv* e *reviews.csv*, respeitando a estrutura definida no modelo de grafo.

O ficheiro *neighbourhoods.csv* foi utilizado para criar os nós *Neighbourhood*, que representam os bairros onde se localizam os alojamentos. Para cada bairro foi armazenado o nome e, quando disponível, o respetivo grupo ou distrito.

De seguida, foi importado o ficheiro *listings.csv*, que contém informação sobre os alojamentos e os seus anfitriões. A partir deste ficheiro foram criados os nós *Host* e *Listing*. Os anfitriões foram identificados através do *host_id*, enquanto os alojamentos foram identificados pelo *id*, convertido no modelo para *listing_id*. Durante esta importação foram também carregados atributos relevantes dos alojamentos, como o nome, localização, tipo de alojamento, preço, número mínimo de noites, número de avaliações, classificação média, disponibilidade anual e bairro.

Nesta fase foram também criadas duas relações fundamentais do grafo: a relação *HOSTS*, que liga cada anfitrião aos alojamentos que disponibiliza, e a relação *IN_NEIGHBOURHOOD*, que associa cada alojamento ao bairro onde está localizado.

O ficheiro *calendar.csv* foi utilizado para criar os nós *CalendarDay*, responsáveis por representar a disponibilidade e o preço de cada alojamento numa determinada data. Em cada nó foram guardadas propriedades como o identificador do alojamento, a data, o estado de disponibilidade, o preço, o ano e o mês. Estes atributos permitem responder às questões relacionadas com evolução temporal, procura, disponibilidade e pagamentos estimados.

Por fim, o ficheiro *reviews.csv* foi utilizado para criar os nós *Guest* e *Review*. Os hóspedes foram identificados através do *reviewer_id*, enquanto as avaliações foram identificadas pelo respetivo *id*. Em cada avaliação foram guardadas informações como a data, o comentário e o identificador do alojamento avaliado. Nesta importação foi ainda criada a relação *WROTE*, que liga cada hóspede à avaliação que escreveu.

Durante o processo de importação foram realizadas várias transformações aos dados. Alguns valores foram convertidos para inteiros, como identificadores e contagens; os preços foram convertidos para valores numéricos; as datas foram convertidas para o tipo *date*; e os valores nulos ou vazios foram tratados através da função *coalesce*. Estas transformações foram necessárias para garantir que os dados ficavam adequados ao modelo em grafo e às consultas analíticas desenvolvidas.

Durante a implementação, verificou-se que os ficheiros *calendar.csv* e *reviews.csv* apresentavam um volume elevado de registos, o que dificultava a importação completa no ambiente local em *Docker*. Para ultrapassar esta limitação, foi importada uma amostra de 25 000 registos de cada um destes ficheiros. Esta opção permitiu validar a estrutura do grafo, criar as relações principais entre as entidades e executar as questões analíticas definidas. Apesar de não ter sido importado o volume total dos ficheiros *calendar.csv* e *reviews.csv*, a estrutura lógica do modelo foi mantida e as consultas puderam ser testadas com dados reais.

```
LOAD CSV WITH HEADERS FROM 'file:///reviews.csv' AS row
WITH row
LIMIT 25000
WHERE coalesce(trim(row.id), '') <> ''
  AND coalesce(trim(row.listing_id), '') <> ''
  AND coalesce(trim(row.reviewer_id), '') <> ''
WITH row,
  toInteger(trim(row.id)) AS reviewId,
  toInteger(trim(row.listing_id)) AS listingId,
  toInteger(trim(row.reviewer_id)) AS reviewerId
WHERE reviewId IS NOT NULL
  AND listingId IS NOT NULL
  AND reviewerId IS NOT NULL
MERGE (g:Guest {reviewer_id: reviewerId})
SET g.reviewer_name = trim(coalesce(row.reviewer_name, ''))
MERGE (r:Review {review_id: reviewId})
SET r.listing_id = listingId,
  r.date = CASE
    WHEN coalesce(trim(row.date), '') = '' THEN NULL
    ELSE date(trim(row.date))
  END,
  r.comments = coalesce(row.comments, '')
MERGE (g)-[:WROTE]->(r);
```

Criação das Relações entre Entidades

Depois da importação dos nós, foi necessário completar as relações entre algumas entidades. Os nós *CalendarDay* e *Review* foram criados inicialmente com o identificador do alojamento associado, mas ainda sem ligação direta ao respetivo nó *Listing*.

Assim, foi criada a relação *HAS_CALENDAR*, que liga cada alojamento aos seus registos diários de disponibilidade e preço. Esta relação permite analisar a evolução da disponibilidade, os preços por data e os dias interpretados como reservados.

Também foi criada a relação *REVIEWS*, que liga cada avaliação ao alojamento avaliado. Com esta relação, tornou-se possível navegar do hóspede para a avaliação e da avaliação para o alojamento, permitindo analisar padrões de comportamento dos hóspedes e volume de avaliações por casa.

Desta forma, o grafo ficou estruturado com as relações principais do domínio: *HOSTS*, *IN_NEIGHBOURHOOD*, *HAS_CALENDAR*, *WROTE* e *REVIEWS*.

```
MATCH (l:Listing)
MATCH (c:CalendarDay {listing_id: l.listing_id})
MERGE (l)-[:HAS_CALENDAR]->(c);
```

```
MATCH (r:Review)
MATCH (l:Listing {listing_id: r.listing_id})
MERGE (r)-[:REVIEWS]->(l);
```

Verificação da Importação

Após a importação dos dados e a criação das relações, foram executadas consultas de verificação para confirmar se a base de dados tinha sido corretamente construída.

Primeiro, foi verificada a existência dos diferentes tipos de nós criados. Esta consulta confirmou a presença dos nós *Neighbourhood*, *Host*, *Listing*, *CalendarDay*, *Review* e *Guest*, correspondentes às entidades definidas no modelo de dados para implementação.

De seguida, foi realizada a verificação das relações existentes no grafo. Esta etapa confirmou a criação das relações *HOSTS*, *IN_NEIGHBOURHOOD*, *HAS_CALENDAR*, *WROTE* e *REVIEWS*, garantindo que os nós estavam corretamente ligados entre si.

Estas verificações permitiram assegurar que o modelo em grafo estava corretamente implementado e apto para responder às questões analíticas definidas para o Momento 3.

```
MATCH (n)
RETURN labels(n) AS tipo, count(n) AS total
ORDER BY tipo;
```

```
MATCH ()-[r]->()
RETURN type(r) AS relacao, count(r) AS total
ORDER BY relacao;
```

Execução das Questões Analíticas

Com a base de dados estruturada e validada, foram executadas as questões analíticas utilizando a linguagem *Cypher*. As consultas permitiram analisar a distribuição dos preços por bairro e tipo de alojamento, a relação entre preço e avaliações, os bairros com maior concentração de casas e procura, o perfil dos anfitriões, a evolução da disponibilidade e do preço ao longo do ano, a estimativa de valores de reserva, a relação entre avaliações e volume de reservas, o impacto das características dos alojamentos no preço e na procura, a distribuição dos pagamentos estimados e os padrões de comportamento dos hóspedes.

A execução destas consultas demonstrou que o modelo em grafo permite navegar diretamente pelas relações entre entidades, evitando junções explícitas como no modelo relacional e sem necessidade de criar tabelas orientadas a cada consulta, como aconteceu em *Cassandra*.

A listagem completa do código utilizado, incluindo a criação das *constraints* e índices, a importação dos ficheiros CSV, a criação das relações e as consultas analíticas, encontra-se no anexo correspondente ao Momento 3.

Síntese de Resultados MySQL

A análise realizada no Momento 1 permitiu obter uma visão clara sobre o mercado de alojamento local em Barcelona, utilizando o modelo relacional implementado no *DBeaver* e as questões analíticas definidas. Os resultados evidenciaram que os preços variam significativamente entre bairros, sendo mais elevados nas zonas centrais e turísticas, e que o tipo de alojamento influencia diretamente o preço, com apartamentos inteiros a apresentarem valores médios superiores aos quartos privados. Verificou-se também que alojamentos com melhores avaliações tendem a praticar preços ligeiramente superiores, embora esta relação varie consoante a localização.

A análise da procura mostrou que os bairros com maior número de alojamentos não são necessariamente os mais reservados, destacando-se as áreas de maior interesse turístico. A partir dos dados de disponibilidade diária foi possível identificar períodos de estadia, permitindo calcular valores estimados de pagamento, que aumentam com a duração da reserva e com a época do ano. Observou-se ainda que muitos anfitriões gerem várias propriedades e que os *superhosts* apresentam maior volume de avaliações e preços médios superiores. Por outro lado, os hóspedes mais ativos tendem a reservar alojamentos mais centrais e melhor avaliados, revelando padrões de comportamento consistentes.

Durante o desenvolvimento do trabalho surgiram algumas dificuldades, sobretudo na identificação dos períodos de estadia, uma vez que o *dataset* não disponibiliza reservas explícitas. Esta limitação foi ultrapassada através da análise de sequências de dias consecutivos marcados como indisponíveis. A limpeza e padronização dos dados, incluindo a conversão de datas e preços, também exigiu atenção adicional, sendo resolvida com tabelas intermédias e transformações *SQL*. No geral, o modelo relacional mostrou-se eficaz para responder às questões analíticas e estabeleceu uma base sólida para os momentos seguintes do projeto.

Síntese de Resultados Cassandra

A implementação realizada no Momento 2 evidencia diferenças estruturais e operacionais significativas relativamente ao modelo relacional desenvolvido no Momento 1. Estas diferenças resultam diretamente dos paradigmas distintos que orientam cada tecnologia e refletem-se tanto na organização dos dados como na forma de responder às questões analíticas definidas.

No modelo relacional, utilizado no M1, os dados encontram-se normalizados e distribuídos por várias tabelas interligadas através de chaves estrangeiras. Este modelo privilegia a integridade referencial e a eliminação de redundância, permitindo a execução de consultas complexas com múltiplas junções. No entanto, tal abordagem torna-se menos eficiente quando aplicada a cenários com grandes volumes de dados e requisitos de desempenho elevados.

Em contraste, a implementação em *Cassandra* no M2 segue um modelo orientado às consultas, no qual a estrutura das tabelas é desenhada especificamente para otimizar o acesso aos dados necessários para cada questão analítica. Assim, as junções e agregações que no M1 eram realizadas dinamicamente passam a ser pré-computadas no *MySQL* e armazenadas de forma desnormalizada em *Cassandra*. Esta abordagem reduz drasticamente o custo computacional das operações de leitura, permitindo que cada consulta aceda diretamente aos dados relevantes.

Outra diferença fundamental reside na definição das chaves primárias. Enquanto no modelo relacional a chave primária identifica unicamente cada registo, em *Cassandra* a chave primária é composta por uma *partition key* e uma *clustering key*, que determinam a distribuição física dos dados e a sua ordenação interna. No M2, estas chaves foram definidas de forma a refletir padrões de acesso específicos, como a análise temporal (partição por ano e mês) ou a análise por bairro (partição por *neighbourhood*), garantindo assim eficiência e escalabilidade.

A necessidade de desnormalização controlada é igualmente uma característica distintiva do M2. Para suportar diferentes questões analíticas, foi necessário criar múltiplas tabelas que replicam parcialmente a mesma informação, mas organizadas segundo diferentes critérios de acesso. Este tipo de redundância, que seria indesejável num modelo relacional, é essencial em *Cassandra* para assegurar tempos de resposta reduzidos e evitar operações de junção, que não são suportadas nativamente.

Em síntese, o M2 demonstra que a transição para *Cassandra* implica uma mudança conceptual: o foco desloca-se da consistência e flexibilidade do modelo relacional para a eficiência e escalabilidade de um sistema distribuído. Embora a organização dos dados seja substancialmente diferente, a estrutura orientada às consultas permite responder às mesmas questões analíticas com maior desempenho.

Síntese de Resultados Neo4j

A implementação do Momento 3 permitiu adaptar o projeto ao modelo orientado a grafos, utilizando o *Neo4j*. Ao contrário do Momento 2, em que as questões analíticas foram agrupadas em tabelas específicas no *Cassandra*, no *Neo4j* foi criado um modelo baseado em nós e relações. Assim, entidades como *Host*, *Listing*, *Neighbourhood*, *CalendarDay*, *Review* e *Guest* passaram a estar diretamente ligadas através de relações como *HOSTS*, *IN_NEIGHBOURHOOD*, *HAS_CALENDAR*, *WROTE* e *REVIEWS*.

No *Cassandra*, a organização dos dados foi feita em função das consultas. Por exemplo, as questões 1 e 2 foram agrupadas numa tabela com atributos estáticos dos alojamentos, enquanto as questões 3, 5, 6 e 9 foram agrupadas numa tabela baseada em dados temporais de disponibilidade e preço. Esta abordagem permitiu responder rapidamente a consultas previamente definidas, mas exigiu desnormalização e criação de tabelas diferentes para grupos de questões.

Em *Neo4j*, as mesmas questões foram respondidas através da navegação pelo grafo. As análises sobre preços por bairro e tipo de alojamento passaram a consultar diretamente os nós *Listing*; as questões sobre disponibilidade, procura e pagamentos estimados passaram a usar a relação *HAS_CALENDAR*; e as questões relacionadas com avaliações e comportamento dos hóspedes passaram a explorar as relações *WROTE* e *REVIEWS*. Desta forma, o *Neo4j* permitiu responder às questões sem criar uma estrutura diferente para cada grupo de consultas.

A principal diferença entre *Cassandra* e *Neo4j* verificou-se, por isso, na forma de modelar os dados. Em *Cassandra*, foi necessário preparar os dados antecipadamente de acordo com cada padrão de leitura. Já em *Neo4j*, a estrutura ficou mais próxima do domínio real do *Airbnb*, permitindo explorar ligações entre anfitriões, alojamentos, bairros, calendário, avaliações e hóspedes de forma mais flexível.

Relativamente aos resultados analíticos, ambos os modelos permitiram responder às questões definidas, mas com abordagens distintas. *Cassandra* mostrou-se mais adequado para consultas diretas e previamente estruturadas, como análises por mês, bairro ou tipo de alojamento. *Neo4j* destacou-se nas questões em que as relações entre entidades são mais relevantes, como o perfil dos anfitriões, a ligação entre avaliações e alojamentos, e os padrões de comportamento dos hóspedes.

Em síntese, o Momento 3 demonstrou que o *Neo4j* complementa a abordagem desenvolvida em *Cassandra*. Enquanto *Cassandra* privilegia a eficiência de leitura através de tabelas orientadas às consultas, *Neo4j* privilegia a análise das relações entre entidades, permitindo uma representação mais natural e flexível do domínio *Airbnb*.



Avaliação do Grupo de Trabalho

Estudante	Avaliação (n=17)	Assinatura Digital
Ana Gomes	16	
Filipe Sousa	13	
Luana Pereira	20	
Nelson Teixeira	16	
Rui Oliveira	20	

Metodologia e critérios definidos para avaliação:

1. **Cumprimento de tarefas atribuídas:** Grau de realização das tarefas que cada elemento assumiu ao longo do trabalho.
2. **Qualidade do trabalho realizado:** Rigor, correção e cuidado na execução das partes atribuídas.
3. **Responsabilidade e autonomia:** Capacidade de resolver problemas, tomar iniciativa e trabalhar de forma independente.
4. **Contributo para o produto final:** Impacto real do trabalho de cada elemento no resultado final do projeto.



Anexos

M1

```
-- Bairro
CREATE TABLE Neighbourhoods (
  neighbourhood VARCHAR(255) NOT NULL,
  neighbourhood_group VARCHAR(255) NOT NULL,
  PRIMARY KEY (neighbourhood)
);
```

```
-- Pessoa (Anfitrião)
CREATE TABLE Hosts (
  host_id BIGINT NOT NULL,
  host_name VARCHAR(255) NOT NULL,
  host_since DATE NULL,
  host_is_superhost TINYINT(1) NULL, -- 1=t, 0=f
  PRIMARY KEY (host_id)
);
```

```
-- Calendário diário (por Casa)
CREATE TABLE Calendar (
  listing_id BIGINT NOT NULL,
  date DATE NOT NULL,
  available TINYINT(1) NOT NULL, -- 1=t, 0=f
  price DECIMAL(10,2) NOT NULL,

  PRIMARY KEY (listing_id, date),
  KEY idx_calendar_date (date),

  CONSTRAINT fk_calendar_listing
  FOREIGN KEY (listing_id) REFERENCES Listings(id)
  ON UPDATE CASCADE
  ON DELETE CASCADE
);
```

```
-- Avaliação
CREATE TABLE Reviews (
  id BIGINT NOT NULL, -- review id
  listing_id BIGINT NOT NULL,
  date DATE NOT NULL,
  reviewer_id BIGINT NOT NULL,
  reviewer_name VARCHAR(255) NOT NULL,
  comments TEXT NULL,

  PRIMARY KEY (id),
  KEY idx_reviews_listing_date (listing_id, date),

  CONSTRAINT fk_reviews_listing
  FOREIGN KEY (listing_id) REFERENCES Listings(id)
  ON UPDATE CASCADE
  ON DELETE CASCADE
);
```

```
-- Casa (Anúncio)
CREATE TABLE Listings (
  id BIGINT NOT NULL, -- listing id
  host_id BIGINT NOT NULL,
  neighbourhood VARCHAR(255) NOT NULL,

  latitude DECIMAL(9,6) NULL,
  longitude DECIMAL(9,6) NULL,

  room_type ENUM('Entire home/apt', 'Private room', 'Shared room', 'Hotel room') NOT NULL,
  property_type VARCHAR(255) NOT NULL,

  accommodates INT NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  minimum_nights INT NOT NULL,

  number_of_reviews INT NOT NULL DEFAULT 0,
  review_scores_rating INT NULL, -- 0..100 ou NULL
  availability_365 INT NOT NULL, -- 0..365

  PRIMARY KEY (id),

  KEY idx_listings_host (host_id),
  KEY idx_listings_neighbourhood (neighbourhood),

  CONSTRAINT fk_listings_host
  FOREIGN KEY (host_id) REFERENCES Hosts(host_id)
  ON UPDATE CASCADE
  ON DELETE RESTRICT,

  CONSTRAINT fk_listings_neighbourhood
  FOREIGN KEY (neighbourhood) REFERENCES Neighbourhoods(neighbourhood)
  ON UPDATE CASCADE
  ON DELETE RESTRICT
);
```



```
-- Bairros
LOAD DATA LOCAL INFILE 'C:/bd2/mysql_data/csv/neighbourhoods.csv'
IGNORE
INTO TABLE Neighbourhoods
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(neighbourhood, neighbourhood_group);
```

```
-- Hosts (a partir de listings.csv)
LOAD DATA LOCAL INFILE 'C:/bd2/mysql_data/csv/listings.csv'
INTO TABLE Hosts
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(host_id, host_name, host_since, @superhost, @d1, @d2, @d3, @d4, @d5)
SET host_is_superhost = IF(@superhost='t',1,0);
```

```
SET FOREIGN_KEY_CHECKS = 0;
LOAD DATA LOCAL INFILE 'C:/bd2/mysql_data/csv/listings_summary.csv'
INTO TABLE Listings
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(
  id,
  @name,
  host_id,
  @host_name,
  @neighbourhood_group,
  neighbourhood,
  latitude,
  longitude,
  room_type,
  price,
  minimum_nights,
  number_of_reviews,
  @last_review,
  @reviews_per_month,
  @calc_count,
  availability_365,
  @reviews_ltm,
  @license
)
SET
  property_type = 'Unknown',
  accommodates = 0,
  review_scores_rating = NULL;
SET FOREIGN_KEY_CHECKS = 1;
```

```
SET FOREIGN_KEY_CHECKS = 0;

LOAD DATA LOCAL INFILE 'C:/bd2/mysql_data/csv/reviews.csv'
INTO TABLE Reviews
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(listing_id, id, date, reviewer_id, reviewer_name, comments);

SET FOREIGN_KEY_CHECKS = 1;
```

M2

Questões 1, 2 e 8:

```
SELECT
  l.neighbourhood,
  l.room_type,
  l.id AS listing_id,
  l.price,
  l.review_scores_rating,
  l.number_of_reviews
FROM Listings l;
```

```
SELECT
  YEAR(c.date) AS year,
  MONTH(c.date) AS month,
  l.accommodates,
  l.room_type,
  l.neighbourhood,
  c.listing_id,
  c.date,
  CASE
    WHEN c.available = 1 THEN 'true'
    WHEN c.available = 0 THEN 'false'
    ELSE NULL
  END AS available,
  l.price
FROM Calendar c
JOIN Listings l ON c.listing_id = l.id;
```

```
USE bd2;
DROP TABLE IF EXISTS listings_by_neighbourhood;
CREATE TABLE IF NOT EXISTS listings_by_neighbourhood (
  neighbourhood TEXT,
  room_type TEXT,
  listing_id BIGINT,
  price DECIMAL,
  review_scores_rating DECIMAL,
  number_of_reviews INT,
  PRIMARY KEY ((neighbourhood), room_type, listing_id)
);

DROP TABLE IF EXISTS listings_features_calendar;
CREATE TABLE IF NOT EXISTS listings_features_calendar (
  year INT,
  month INT,
  accommodates INT,
  room_type TEXT,
  neighbourhood TEXT,
  listing_id BIGINT,
  date DATE,
  available BOOLEAN,
  price DECIMAL,
  PRIMARY KEY ((year, month), accommodates, room_type, listing_id, date)
);

SELECT COUNT(*) FROM listings_by_neighbourhood;
SELECT COUNT(*) FROM listings_features_calendar WHERE year = 2026 AND month = 2;
```

Questões 3, 5, 6 e 9:

```
SELECT
  YEAR(c.date) AS year,
  MONTH(c.date) AS month,
  l.neighbourhood,
  l.room_type,
  c.listing_id,
  c.date,
  CASE
    WHEN c.available = 1 THEN 'true'
    WHEN c.available = 0 THEN 'false'
    ELSE NULL
  END AS available,
  l.price,
  l.minimum_nights
FROM Calendar c
JOIN Listings l
ON c.listing_id = l.id;
```

```
USE bd2;

DROP TABLE IF EXISTS listings_calendar;
CREATE TABLE IF NOT EXISTS listings_calendar(
  year INT,
  month INT,
  neighbourhood TEXT,
  room_type TEXT,
  listing_id BIGINT,
  date DATE,
  available BOOLEAN,
  price DECIMAL,
  minimum_nights INT,
  PRIMARY KEY ((year, month), neighbourhood, room_type, listing_id, date)
);

SELECT *
FROM listings_calendar
WHERE year = 2026 AND month = 2
LIMIT 10;

SELECT COUNT(*) FROM listings_calendar;
```


Questão 4:

```
SELECT
    h.host_id,
    h.host_name,
    h.host_is_superhost AS superhost,
    COUNT(DISTINCT l.id) AS total_casas,
    COUNT(r.id) AS total_avaliacoes
FROM Hosts h
LEFT JOIN Listings l
    ON h.host_id = l.host_id
LEFT JOIN Reviews r
    ON l.id = r.listing_id
GROUP BY
    h.host_id,
    h.host_name,
    h.host_is_superhost
ORDER BY
    total_casas DESC;
```

```
USE bd2;

DROP TABLE IF EXISTS hosts_summary;

CREATE TABLE IF NOT EXISTS hosts_summary (
    host_id BIGINT,
    host_name TEXT,
    superhost BOOLEAN,
    total_casas INT,
    total_avaliacoes INT,
    PRIMARY KEY (host_id)
);

SELECT * FROM hosts_summary WHERE host_id = 721378;

SELECT * FROM hosts_summary LIMIT 50;
```

Questões 7 e 10:

```
SELECT
    r.reviewer_id,
    r.reviewer_name,
    r.listing_id,
    l.room_type,
    l.neighbourhood,
    rr.total_reviews_reviewer,
    rl.total_reviews_listing,
    c.total_reservas_listing
FROM Reviews r
JOIN Listings l ON r.listing_id = l.id
LEFT JOIN (
    SELECT reviewer_id, COUNT(*) AS total_reviews_reviewer
    FROM Reviews
    GROUP BY reviewer_id
) rr ON rr.reviewer_id = r.reviewer_id
LEFT JOIN (
    SELECT listing_id, COUNT(*) AS total_reviews_listing
    FROM Reviews
    GROUP BY listing_id
) rl ON rl.listing_id = r.listing_id
LEFT JOIN (
    SELECT listing_id, COUNT(*) AS total_reservas_listing
    FROM Calendar
    WHERE available = 0
    GROUP BY listing_id
) c ON c.listing_id = r.listing_id
ORDER BY r.reviewer_id, r.listing_id;
```

```
USE bd2;

DROP TABLE IF EXISTS listings_reviews_reservas;

CREATE TABLE IF NOT EXISTS listings_reviews_reservas (
    listing_id BIGINT,
    host_id BIGINT,
    neighbourhood TEXT,
    room_type TEXT,
    total_avaliacoes INT,
    total_reservas INT,
    PRIMARY KEY (listing_id)
);

SELECT COUNT(*) FROM reviews_reservas_comportamento;
```


M3

```
//criação de constraints e índices
CREATE CONSTRAINT neighbour_name IF NOT EXISTS
FOR (n:Neighbourhood) REQUIRE n.name IS UNIQUE;

CREATE CONSTRAINT host_id IF NOT EXISTS
FOR (h:Host) REQUIRE h.host_id IS UNIQUE;

CREATE CONSTRAINT listing_id IF NOT EXISTS
FOR (l:Listing) REQUIRE l.listing_id IS UNIQUE;

CREATE CONSTRAINT review_id IF NOT EXISTS
FOR (r:Review) REQUIRE r.review_id IS UNIQUE;

CREATE CONSTRAINT reviewer_id IF NOT EXISTS
FOR (g:Guest) REQUIRE g.reviewer_id IS UNIQUE;

CREATE CONSTRAINT calendar_key IF NOT EXISTS
FOR (c:CalendarDay) REQUIRE (c.listing_id, c.date) IS UNIQUE;

CREATE INDEX listing_neighbourhood IF NOT EXISTS
FOR (l:Listing) ON (l.neighbourhood);

CREATE INDEX listing_room_type IF NOT EXISTS
FOR (l:Listing) ON (l.room_type);

CREATE INDEX calendar_year_month IF NOT EXISTS
FOR (c:CalendarDay) ON (c.year, c.month);

CREATE INDEX review_date IF NOT EXISTS
FOR (r:Review) ON (r.date);
```

```
//csv
//importação do neighbourhood
LOAD CSV WITH HEADERS FROM 'file:///neighbourhoods.csv' AS row
WITH row
WHERE row.neighbourhood IS NOT NULL AND trim(row.neighbourhood) <> ''
MERGE (n:Neighbourhood {name: trim(row.neighbourhood)})
SET n.neighbourhood_group = trim(coalesce(row.neighbourhood_group, ''));
```

```
//importação parcial do calendar
LOAD CSV WITH HEADERS FROM 'file:///calendar.csv' AS row
WITH row
LIMIT 25000
WHERE coalesce(trim(row.listing_id), '') <> ''
| AND coalesce(trim(row.date), '') <> ''
WITH row,
| | | toInteger(trim(row.listing_id)) AS listingId,
| | | date(trim(row.date)) AS calendarDate
WHERE listingId IS NOT NULL
MERGE (c:CalendarDay {listing_id: listingId, date: calendarDate})
SET c.available = CASE
| | | WHEN toLower(trim(coalesce(row.available, 'f'))) IN ['t', 'true'] THEN true
| | | ELSE false
END,
c.price = CASE
| | | WHEN coalesce(trim(row.price), '') = '' THEN 0.0
| | | ELSE toFloat(replace(replace(trim(row.price), '$', ''), ',', ''))
END,
c.year = toInteger(substring(trim(row.date), 0, 4)),
c.month = toInteger(substring(trim(row.date), 5, 2));
```

```
//importação dos listings
LOAD CSV WITH HEADERS FROM 'file:///listings.csv' AS row
WITH row
WHERE row.id IS NOT NULL AND trim(row.id) <> ''
MERGE (h:Host {host_id: toInteger(row.host_id)})
SET h.host_name = trim(coalesce(row.host_name, '')),
h.host_since = CASE
WHEN row.host_since IS NULL OR trim(row.host_since) = '' THEN NULL
ELSE date(row.host_since)
END,
h.host_is_superhost = CASE
WHEN toLower(trim(coalesce(row.host_is_superhost, 'f'))) = 't' THEN true
ELSE false
END
MERGE (n:Neighbourhood {name: trim(coalesce(row.neighbourhood_cleansed, row.neighbourhood, 'Desconhecido'))})
ON CREATE SET n.neighbourhood_group = trim(coalesce(row.neighbourhood_group_cleansed, row.neighbourhood_group, ''))
MERGE (l:Listing {listing_id: toInteger(row.id)})
SET l.name = trim(coalesce(row.name, '')),
l.latitude = CASE WHEN row.latitude IS NULL OR trim(row.latitude) = '' THEN NULL ELSE toFloat(row.latitude) END,
l.longitude = CASE WHEN row.longitude IS NULL OR trim(row.longitude) = '' THEN NULL ELSE toFloat(row.longitude) END,
l.room_type = trim(coalesce(row.room_type, '')),
l.property_type = trim(coalesce(row.property_type, 'Unknown')),
l.accommodates = CASE WHEN row.accommodates IS NULL OR trim(row.accommodates) = '' THEN 0 ELSE toInteger(row.accommodates) END,
l.price = CASE
WHEN row.price IS NULL OR trim(row.price) = '' THEN 0.0
ELSE toFloat(replace(replace(replace(trim(row.price), '$', ''), ',', ''), ' ', ''))
END,
l.minimum_nights = CASE WHEN row.minimum_nights IS NULL OR trim(row.minimum_nights) = '' THEN 0 ELSE toInteger(row.minimum_nights) END,
l.number_of_reviews = CASE WHEN row.number_of_reviews IS NULL OR trim(row.number_of_reviews) = '' THEN 0 ELSE toInteger(row.number_of_reviews) END,
l.review_scores_rating = CASE WHEN row.review_scores_rating IS NULL OR trim(row.review_scores_rating) = '' THEN NULL ELSE toFloat(row.review_scores_rating) END,
l.availability_365 = CASE WHEN row.availability_365 IS NULL OR trim(row.availability_365) = '' THEN 0 ELSE toInteger(row.availability_365) END,
l.neighbourhood = trim(coalesce(row.neighbourhood_cleansed, row.neighbourhood, 'Desconhecido'))
MERGE (h)-[:HOSTS]->(l)
MERGE (l)-[:IN_NEIGHBOURHOOD]->(n);
```

```
//importação parcial das reviews
LOAD CSV WITH HEADERS FROM 'file:///reviews.csv' AS row
WITH row
LIMIT 25000
WHERE coalesce(trim(row.id), '') <> ''
  AND coalesce(trim(row.listing_id), '') <> ''
  AND coalesce(trim(row.reviewer_id), '') <> ''
WITH row,
  toInteger(trim(row.id)) AS reviewId,
  toInteger(trim(row.listing_id)) AS listingId,
  toInteger(trim(row.reviewer_id)) AS reviewerId
WHERE reviewId IS NOT NULL
  AND listingId IS NOT NULL
  AND reviewerId IS NOT NULL
MERGE (g:Guest {reviewer_id: reviewerId})
SET g.reviewer_name = trim(coalesce(row.reviewer_name, ''))
MERGE (r:Review {review_id: reviewId})
SET r.listing_id = listingId,
  r.date = CASE
    WHEN coalesce(trim(row.date), '') = '' THEN NULL
    ELSE date(trim(row.date))
  END,
  r.comments = coalesce(row.comments, '')
MERGE (g)-[:WROTE]->(r);
```

```
//criação das relações entre entidades
MATCH (l:Listing)
MATCH (c:CalendarDay {listing_id: l.listing_id})
MERGE (l)-[:HAS_CALENDAR]->(c);

MATCH (r:Review)
MATCH (l:Listing {listing_id: r.listing_id})
MERGE (r)-[:REVIEWS]->(l);

// verificação dos nós criados
MATCH (n)
RETURN labels(n) AS tipo, count(n) AS total
ORDER BY tipo;

// verificação das relações criadas
MATCH ()-[r]->()
RETURN type(r) AS relacao, count(r) AS total
ORDER BY relacao;
```

Referências Bibliográficas

Inside Airbnb. (2024). *Inside Airbnb: Get the data*. <https://insideairbnb.com/get-the-data/>

Inside Airbnb. (2024). *Barcelona – Listings, Calendar and Reviews datasets*. Inside Airbnb. <https://insideairbnb.com/barcelona/>